

Sesión 2 de Ako — resumen técnico

30 de septiembre de 2026, 11:16 – ~13:35. Objetivo cumplido: la capa de datos del nivel 1 compila y la precarga se ve en el Database Inspector. Commits `805593a`, `d23b193` (código) y `01293fe` (ficha).

1. Repaso de la S1

- **Gradle**: sistema de construcción. Resuelve y descarga las dependencias, compila Kotlin y los recursos, ejecuta KSP (el generador de código de Room) y empaqueta el APK. `app/build.gradle.kts` declara el módulo (`minSdk`, dependencias).
- **Catálogo de versiones** (`gradle/libs.versions.toml`): `[versions]` + `[libraries]` + `[plugins]`. Centraliza las coordenadas y versiones de las dependencias (p. ej. `room = "2.8.5"`); los `build.gradle.kts` las citan como `libs.androidx.room.runtime`. No contiene la versión de la app.
- `minSdk 26`: `PBKDF2WithHmacSHA256` está disponible en la API 26 sin librerías externas; cobertura ~98,4 %.

2. Conceptos de base de datos relacional

Tabla, fila, clave primaria (PK), clave foránea (FK), `onDelete = RESTRICT` (impide borrar la fila padre referenciada) frente a `CASCADE` (borra las hijas), índice único e `@Entity` de Room (anotación procesada por KSP que genera el esquema SQLite).

3. Decisiones de la sesión (todas elegidas por Daniel, P112)

Decisión	Elegida	Alternativas	Fuente
P113 · enums en la BD	Conversor de enums incorporado en Room (desde 2.3.0; guarda el <code>name</code> como TEXT); sin <code>Converters</code>	<code>@TypeConverter</code> propio por nombre · por <code>ordinal</code>	developer.android.com/jetpack/androidx/releases/room#2.3.0
P115 · alcance de la BD	Solo el nivel 1: 7 tablas , ninguna CASCADE	14 tablas (spec) · quitar solo modificadores	developer.android.com/training/data-storage/room/migrating-db-versions
P116 · instancia única	<code>val db by lazy { Room.databaseBuilder(...).build() }</code> en la clase <code>Application</code>	<code>companion object</code> con <code>@Volatile</code> + <code>synchronized</code> (codelab) · <code>object</code> relleno en <code>onCreate</code>	developer.android.com/training/data-storage/room · kotlinlang.org/docs/delegated-properties.html#lazy-properties · codelab «Persist data with Room»
P117 · nombre de la <code>Application</code>	<code>EntradaAko</code> (propuesta de Daniel)	<code>AccesoBaseDatos</code> · <code>AkoApplication</code> · <code>Central</code>	codelab (<code>InventoryApplication</code>)
P118 · apertura de la BD	<code>EntradaAko.onCreate</code> abre la BD en una corrutina (<code>openHelper.writableDatabase</code>) y así dispara <code>Callback.onCreate</code> en el primer arranque	Línea provisional en <code>MainActivity</code> · no forzar	discuss.kotlinlang.org/t/room-oncreate-isnt-called-after-databasebuilder-build/15124

Reglas de alcance (en `CLAUDE.md`): el orden es prototipo (nivel 1) → memoria → nivel 2 solo si sobra tiempo y por incrementos enteros (sustituye a P3); nada de código especulativo; el diseño del nivel 2 se conserva; lo decidido en Claude

Code prevalece sobre el Project y se anota en `decisiones-code.md` §6.

4. Lo construido

Paquetes: `datos/{entidades,dao,repositorios}`, `dominio/modelos`, `seguridad`, `ui`. `dominio` es Kotlin puro (sin `android.*`), testeable con JUnit en la JVM.

EstadoComanda (`PENDIENTE`, `PAGADA`, `ANULADA`): enum para que el compilador detecte valores inválidos.

Entidades (columnas en `snake_case` con `@ColumnInfo`, PK Long autogenerada):

Tabla	Campos	Restricciones
<code>categoria</code>	<code>id</code> , <code>nombre</code> , <code>imagen?</code> , <code>orden</code> , <code>activo</code> , <code>es_por_defecto</code>	índice único <code>nombre</code>
<code>producto</code>	<code>id</code> , <code>categoria_id</code> , <code>numero</code> , <code>nombre</code> , <code>descripcion?</code> , <code>precio_centimos</code> , <code>imagen?</code> , <code>activo</code>	índice único <code>numero</code> ; FK → <code>categoria</code> RESTRICT; índice <code>categoria_id</code>
<code>alergeno</code>	<code>id</code> , <code>nombre</code>	—
<code>producto_alergeno</code>	<code>producto_id</code> , <code>alergeno_id</code>	PK compuesta ; 2 FK RESTRICT; índice <code>alergeno_id</code>
<code>mesa</code>	<code>id</code> , <code>numero</code>	índice único <code>numero</code>
<code>comanda</code>	<code>id</code> , <code>mesa_id</code> , <code>estado</code> , <code>fecha_creacion</code> , <code>fecha_cierre?</code>	FK → <code>mesa</code> RESTRICT; índice <code>mesa_id</code>
<code>linea_comanda</code>	<code>id</code> , <code>comanda_id</code> , <code>producto_id</code> , <code>cantidad</code> , <code>precio_unitario_centimos</code> , <code>nombre_producto</code>	2 FK RESTRICT; índices

Los índices sobre columnas FK los añadió Claude porque, sin ellos, Room avisa (evitan recorrer la tabla entera). Principios aplicados: importes en `Int` de céntimos (sin error de coma flotante); lo calculable no se guarda (ocupación de la mesa R3, total R10); la línea congela nombre y precio (R14); las fechas son epoch en milisegundos (`Long`).

AppDatabase: `@Database(entities = [7], version = 1, exportSchema = false)`, clase abstracta cuya implementación (`AppDatabase_Impl`) genera KSP. Expone 4 DAOs.

DAOs (solo lo que usa la precarga; la S3 añade las consultas): `CategoriaDao.insertar(): Long`, `ProductoDao.insertar(): Long`, `MesaDao.insertarTodas(List)`, `PrecargadosDao.insertarAlergenos(List)`. Todos `suspend`: Room prohíbe acceder a la BD desde el hilo principal.

EntradaAko : Application (registrada con `android:name=".EntradaAko` en el manifiesto): guarda la única instancia con `by lazy`, que por defecto es `SYNCHRONIZED` (thread-safe). Su `onCreate` lanza `CoroutineScope(Dispatchers.IO).launch { db.openHelper.writableDatabase }`.

strings.xml: 14 alérgenos (`alergeno_01_gluten` ... `alergeno_14_moluscos`, en el orden del anexo II del Reglamento UE 1169/2011), `precarga_categoria_por_defecto`, `precarga_categoria_ejemplo` y `precarga_plato_ejemplo` (RNF-19: nada de textos escritos en el código).

Precarga : RoomDatabase.Callback (P16): `onCreate` («se llama cuando la base de datos se crea por primera vez») lanza una corrutina en `Dispatchers.IO` que llama a `suspend fun cargar(db: AppDatabase)`. `cargar` inserta, en orden: Otros (`esPorDefecto = true`, `orden = 0`, P44), las mesas 1–60, los 14 alérgenos, y Bebidas (`orden = 1`) con `1 · Agua · 150` céntimos, usando el `id` devuelto por `insertar`. Que `cargar` reciba la BD permite que las pruebas de la S11 la llamen con una base de datos en memoria.

5. Verificación

- `compileDebugKotlin`, `assembleDebug` y `testDebugUnitTest` en verde.
- App desinstalada (`adb uninstall yunkang.ako`) y reinstalada. En el Database Inspector y en `ako.db` extraído con `adb exec-out run-as: categoria 2` filas, producto 1 (`categoria_id = 2`), `mesa 60`, `alergeno 14`; `producto_alergeno`, `comanda` y `linea_comanda` vacías. `room_master_table` es interna de Room (hash del esquema).
- RF-50 pasa a *implementado, no probado* (P-M-29 se ejecuta en la S9; su paso de la tabla `etiqueta` se quita por P115).

6. Problemas y soluciones

- Paquetes creados fuera de `yunkang.ako` → se rehicieron con clic derecho sobre el paquete correcto.
- `Comanda.java` con código Kotlin → Android Studio elige el compilador por la extensión; se renombró a `.kt`.
- Shift + F6 renombró la clase equivocada (`LineaComanda.kt.kt`) → se restauró; el refactor actúa sobre la selección o el cursor.
- DAOs en `datos/dao/datos/` y `CategoriaDao` en `dao.kt` → se movieron y renombraron; el `package` interno ya era correcto.

7. Para defender en el vídeo

- 7 tablas del nivel 1 (carta 4, sala 1, pedidos 2) y por qué no 14: alcance del prototipo, sin código especulativo.
- Todo RESTRICT: en Ako nada se borra, se elimina con `activo = false`.
- Enum frente a String: error en compilación, no en ejecución.
- Única instancia con `by lazy` en la `Application`.
- Precarga con `Callback` + corrutina, y por qué se abre la BD al arrancar.
- Céntimos en `Int`, línea congelada, estado calculado.

Siguiente: S3

DAOs completos (consultas del diagrama, `ComandaDao` nuevo) y dominio puro: `Carrito`, `LineaCarrito`, `Calculadora`, `Validacion`, `Hash`; pruebas P-C-01 a P-C-05 y P-C-09.